

Learning to Navigate in a Stochastic and Uncertain Environment with Different Robot Dynamics

Anonymous Author(s)

Affiliation

Address

email

Abstract: Multi-robot navigation in real-world settings is inherently *stochastic* and *partially predictable*: wind perturbs motion, actuators do not match nominal commands, and onboard sensing is noisy. Yet many reinforcement learning (RL) navigation benchmarks either omit these effects or treat them informally, making it hard to compare algorithms under realistic uncertainty and across heterogeneous platforms. This paper studies learning-based multi-robot navigation under such disturbances while explicitly contrasting *two* continuous-control dynamics families: planar omnidirectional aerial vehicles modeled as force-driven point masses, and nonholonomic wheeled ground robots modeled with bicycle kinematics in cluttered workspaces. We retain a dynamics-agnostic constrained-optimization view of ROI coverage with collision avoidance, derive a corresponding finite-horizon Markov decision process (MDP) reward decomposition, and summarize an equivalence-style argument linking the navigation objective to optimal MDP behavior under idealized full-state observability. We then describe an open, Gymnasium-compatible implementation that augments this idealization with stochastic wind, action noise, observation noise, optional domain randomization, and ablation switches for rewards, observations, and uncertainty modes. Using Stable-Baselines3 and SB3-Contrib, we benchmark six standard deep RL algorithms with default and Optuna-tuned hyperparameters across *ten* randomized aerial field layouts and *eight* wheeled layouts. Finally, we visualize aerial policies in a CoppeliaSim agricultural scene and analyze wind-strength sensitivity; a companion line of work studies selective *spraying* under richer deposition and belief-state structure.

Keywords: CoRL, Robots, Learning

1 Introduction

Developing robots that can plan and navigate in complex, uncertain environments is a fundamental problem in artificial intelligence. Reinforcement learning (RL) is a widely used paradigm for such tasks [1, 2, 3, 4, 5]. Progress in multi-robot continuous control, however, depends not only on algorithms but also on *environments* that expose stochasticity, sensor limitations, and heterogeneous dynamics in reproducible ways [6, 7, 8, 9, 10]. Once formulated as Markov (or partially observable Markov) decision processes, such environments become shared testbeds for gradient-based RL and for derivative-free alternatives such as Augmented Random Search (ARS) [8], evolution strategies [11], and other black-box optimization techniques [12]. Unfortunately, many RL benchmark papers omit a complete MDP description [13, 14] or provide only informal reward engineering for navigation [1, 2, 3, 4, 5], and they seldom evaluate the *same* training stack under controlled uncertainty factors across *different* robot models.

In this paper, we center the question: *how do standard deep RL methods behave when multi-robot navigation is deliberately made stochastic and uncertain, and when the robot dynamics differ sub-*

stantially between platforms? Concretely, we pair (i) a principled optimization-to-MDP construction for ROI visitation with collision avoidance, with (ii) a public codebase that instantiates *two* dynamics classes—aerial point-mass UAVs and wheeled bicycle robots—under configurable wind volatility, actuator noise, observation noise, and domain randomization, all behind a unified training interface. We benchmark common gradient-based algorithms (A2C [15], TRPO [16], PPO [17], TQC [18], CrossQ [19]) together with ARS [8], comparing Stable-Baselines3 defaults against Optuna-selected hyperparameters [20, 21]. Training and evaluation use Gymnasium [22] with two-dimensional Pygame visualization for *both* platforms; for UAVs we additionally export policies to a CoppeliaSim scene for qualitative inspection in an agricultural-style setting. Importantly, the Gymnasium task studied here is *target visitation / coverage* (binary ROI completion), whereas our parallel manuscript treats cooperative *precision spraying* with deposition uncertainty and a partially observable formalization.

While multi-robot navigation has previously been formulated within RL, few works simultaneously (a) articulate an optimization-aligned MDP specification, (b) stress-test identical algorithmic baselines under multi-source uncertainty, and (c) keep the same experimental switches across qualitatively different dynamics. Relative to our earlier emphasis on “environment design” as the headline, the present title highlights *learning under stochasticity* and *cross-dynamics* evaluation. To summarize, the contributions are:

- **Formal design template:** We present a finite-horizon MDP reward decomposition for multi-robot ROI coverage with collision penalties and redundant-motion costs, together with an analytical linkage to a constrained navigation optimization problem under idealized assumptions (Section 4–5).
- **Stochastic, uncertainty-aware benchmark:** We release Gymnasium environments with stochastic wind, action noise, observation noise, optional domain randomization, and ablations over reward terms, observation channels, and uncertainty modes, enabling controlled studies of robustness (Section 6).
- **Cross-dynamics evaluation:** Using one training codebase, we evaluate six deep RL algorithms on *ten* randomized aerial fields and *eight* wheeled obstacle layouts, reporting episodic return statistics under default and tuned hyperparameters.
- **Simulation and transfer-oriented analysis:** We visualize aerial behaviors in CoppeliaSim, perform wind-strength sensitivity at inference time, and discuss observation-noise stress tests; a separate companion paper focuses on selective spraying with richer physical models.

2 Motivation and Related Work

Over the past two decades, numerous methods have been developed for multi-robot navigation and control [23, 24, 25]. Classical planning approaches such as A* [26], RRT [27], RRT* [28], PSO [23], and RA-MCPP [25] rely on explicit modeling of path-planning heuristics or global optimization strategies. While effective in structured settings, these methods often become computationally expensive in high-dimensional continuous state and action spaces, particularly when multiple robots and dynamic environments are involved. As a result, scalable multi-robot navigation in complex environments remains a fundamental challenge in robotics [29, 30, 31, 32, 33].

Reinforcement Learning (RL) has emerged as a promising alternative, especially for problems requiring real-time decision-making and adaptation in high-dimensional spaces [34, 1, 2, 35, 36, 9, 30]. Early RL-based navigation works [34, 35] often considered discrete or simplified state-action spaces. More recent approaches [1, 2, 3, 4, 5] formulate navigation objectives as optimization problems and derive corresponding Markov Decision Processes (MDPs). However, these formulations typically lack formal guarantees establishing equivalence between the navigation objective and the optimal policy. In addition, many works assume fixed robot dynamics, rely heavily on reward heuristics, or introduce customized RL architectures tailored to specific environments. Beyond navigation, several

benchmark environments have been proposed to bridge RL and real-world challenges. Dulac-Arnold *et al.* [9] introduced a suite of practical RL challenges and corresponding MDP formulations, but none address multi-robot navigation, and formal correctness analyses remain limited. Large-scale environments such as Google Research Football [13] and CityFlow [14] provide realistic testbeds but do not offer formal guarantees relating task objectives to optimal policies. Furthermore, the need for reproducible and extensible baselines in continuous control has been emphasized by Mania *et al.* [8], who showed that algorithmic performance can vary significantly in high-dimensional settings.

In contrast to prior works, we focus on the principled construction of the RL environment itself *and* on exposing stochasticity in a tunable, reproducible way. We provide a complete MDP specification for multi-robot navigation and formally prove that, under a finite-horizon setting and appropriate reward design, the optimal MDP policy is equivalent to the solution of the underlying constrained optimization problem in the idealized setting of Section 5. The accompanying software extends this idealization with stochastic disturbances and noisy observations for empirical RL studies. Moreover, our framework accommodates arbitrary robot dynamics through the generic simulator map $\text{SIM}(\cdot)$ and can be solved using standard state-of-the-art RL algorithms without task-specific architectural modifications. By isolating the contribution to environment design and validating it across diverse dynamics and algorithms, we position our framework as both theoretically grounded and practically extensible for high-dimensional multi-robot systems.

3 Preliminaries

Let $\mathbb{R}, \mathbb{R}^+, \mathbb{N}, \mathbb{Z}$ be the set of reals, positive reals, naturals, and integers, respectively. For a set S , let $|S|$ and $\mathbb{P}(S)$ denote its length and power set, respectively. Given two real numbers $x, y \in \mathbb{R}$, let $[x, y] = \{i \in \mathbb{R} \mid x \leq i \leq y\}$ denote the set of all real numbers from x to y . Let Y be a continuous random variable with probability density function f . The expected value (or expectation) is defined as, $\mathbb{E}[Y] = \int_{-\infty}^{\infty} yf(y)dy$. For a set of samples S , let $\Delta(S)$ denote the set of probability distributions over S . For a nonempty set X , a collection $\mathcal{A} \subseteq \mathcal{P}(X)$ is called σ -algebra [37] on X if it satisfies $X \in \mathcal{A}, \forall A_i \in \mathcal{A} : X \setminus A_i \in \mathcal{A}$ and $\cup_i^\infty A_i \in \mathcal{A}$. Let $(X, \mathcal{A})$ and $(Y, \mathcal{B})$ be measurable spaces where X and Y are sets and $\mathcal{A}$ and $\mathcal{B}$ are σ algebras of subsets of X and Y . A function $f : X \rightarrow Y$ is called measurable if $\forall B \in \mathcal{B} : f^{-1}(B) \in \mathcal{A}$. In RL, a policy $\pi : S \rightarrow \Delta(A)$ is a function that maps a state $s \in S$ to a probability distribution over the set of actions A . The action-value or Q-value of a policy π at state s and action a is defined as the expected cumulative reward from following the policy after taking the action a starting from s and written as, $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a]$ where $r_t \in \mathbb{R}^+$ is the reward at time t and $\gamma \in [0, 1]$ is the discount factor. Following the notations and definitions from Choset *et al.* [38], we define the multi-robot navigation problem in the next section.

4 Problem Formulation

Consider a group of n mobile robots $\mathcal{M} = \{\mathcal{M}_1, \dots, \mathcal{M}_n\}$ operating in a d -dimensional workspace $\mathcal{W} \subseteq \mathbb{R}^d$ containing static obstacles $\mathcal{O}$ and m regions of interests (ROIs) $\mathcal{G} = \{\mathcal{G}_1, \dots, \mathcal{G}_m\}$. Let $\mathcal{W}_{\mathcal{O}_j}$ be the region occupied by obstacle j , and $\cup_j \mathcal{W}_{\mathcal{O}_j}$ is the total region covered by all obstacles in the workspace. Therefore, the free workspace is the region $\mathcal{W}^{\text{free}} = \mathcal{W} \setminus \cup_j \mathcal{W}_{\mathcal{O}_j}$. From Plaku *et al.* [?], each robot model $\mathcal{M}_i = \{\mathcal{P}_i, \mathcal{Q}_i, \mathcal{S}_i, s_i^{\text{init}}, \mathcal{U}_i, f_i\}$ is defined in terms of its shape $\mathcal{P}_i$, configuration space $\mathcal{Q}_i$, state space $\mathcal{S}_i$, initial state $s_i^{\text{init}} \in \mathcal{S}_i$, control space $\mathcal{U}_i$ and dynamics f_i . Given the shape $\mathcal{P}_i$ for robot i , let $\mathcal{P}_i(q_i) \subset \mathcal{W}$ and $\mathcal{P}_i(s_i) \subset \mathcal{W}$ denote the regions occupied by the robot in the workspace $\mathcal{W}$ at configuration $q_i \in \mathcal{Q}_i$ and state $s_i \in \mathcal{S}_i$ respectively. The configuration space occupied by all obstacles for the robot i is, $\mathcal{Q}_i^{\text{obs}} = \{q_i \in \mathcal{Q}_i \mid \exists j : \mathcal{P}_i(q_i) \cap \mathcal{W}_{\mathcal{O}_j} \neq \emptyset\}$. The collision configuration space for robot i is the set of configurations that collide with other robots in the workspace:

$$\mathcal{Q}_i^{\text{col}} = \{q_i \in \mathcal{Q}_i \mid \exists j \neq i : (\mathcal{P}_i(q_i) \cap \mathcal{P}_j(q_j) \neq \emptyset)\} \cup \mathcal{Q}_i^{\text{obs}} \quad (1)$$

134 Since the robots are moving, let $Q_i^{col}(t)$ denote the collision configuration for robot i at time t .
 135 Therefore, the free configuration space for the robot i at time t is, $Q_i^{free}(t) = Q_i \setminus Q_i^{col}(t)$. The
 136 state space $S_i = Q_i \times \mathcal{V}_i$ of robot i augments the configuration space Q_i and motion space $\mathcal{V}_i$,
 137 where $\mathcal{V}_i$ contains the motion parameters such as steering angle, velocity, heading direction, etc.
 138 The kinematics and dynamics of each robot i is described as follows:

$$\dot{q}_i = f_i(q_i, u_i), q_i \in Q_i, u_i \in \mathcal{U}_i \quad (\text{kinematics}) \quad (2)$$

$$\dot{s}_i = f_i(s_i, u_i), s_i \in S_i, u_i \in \mathcal{U}_i \quad (\text{dynamics}) \quad (3)$$

139 A new state $s_i^{new} \in S_i$ can be reached after a small time step dt by numerically integrating f_i by a
 140 function:

$$s_i^{new} \leftarrow \text{SIM}(s_i, u_i, f_i, dt) \quad (4)$$

141 Let $\zeta_i : [0, T] \rightarrow S_i$ denote a trajectory function for robot i for time period $[0, T]$. Let $\text{GS}(\zeta_i(\tau))$
 142 denote the ROIs visited by trajectory ζ_i from time 0 to τ , i.e.,

$$\text{GS}(\zeta_i(\tau)) = \{g \mid g \in \mathcal{G} \wedge \exists t \in [0, \tau] : \mathcal{P}_i(\zeta_i(t)) \cap g \neq \emptyset\} \quad (5)$$

143 We define the multi-robot navigation problem as a constrained optimization problem. Let us define
 144 a simple cost function $\mathcal{C}_i : S_i \rightarrow \{c_0, c_1\}$ where $\mathcal{C}_i(s_i) = c_0$ if state s_i is not visited before
 145 and $\mathcal{C}(s) = c_1$ if s is visited before by robot i , and $c_0 < c_1$ (visited locations has higher costs).
 146 The objective is to generate n trajectories $\zeta_1, \dots, \zeta_n$ for the robots by applying n control signals
 147 $u_1, \dots, u_n$ that solves the optimization problem:

$$\min_{\zeta_1, \dots, \zeta_n} \left(\sum_{i=1}^n \int_{t=0}^T \mathcal{C}_i(\zeta_i(t)) dt \right) \quad (6)$$

subject to,

$$(i) \quad \bigcup_{i=1}^n \text{GS}(\zeta_i(T)) = \mathcal{G} \quad (7)$$

$$(ii) \quad \forall i \in \{1, \dots, n\}, \forall t \in [0, T] : \quad \zeta_i(t) = (q_i, v_i), q_i \in Q_i^{free}(t), v_i \in \mathcal{V}_i \quad (8)$$

$$(iii) \quad \forall i \in \{1, \dots, n\}, \forall t \in [0, T] : \quad \zeta_i(t + dt) \leftarrow \text{SIM}(\zeta_i(t), u_i, f_i, dt) \quad (9)$$

148 Here, the first constraint ensures that all regions of interest or goals are visited. The second constraint
 149 ensures that there are no collisions. The third constraint ensures that each trajectory is dynamically
 150 feasible.

151 5 Environment Design

152 The above optimization problem can be represented as a reinforcement learning (RL) problem. We
 153 define a Markov Decision Process (MDP) $M = (S, A, \mathcal{T}, R, \gamma)$ for the multi-robot navigation prob-
 154 lem where: $\bullet S = S_1 \times \dots \times S_n \times \{0, 1\}^{|\mathcal{G}|}$ is the set of states defined as a joint state space of n
 155 robots, and a bitmask $b_t : \mathcal{G} \rightarrow \{0, 1\}$ that encodes how many regions of interest are visited at time
 156 t . Therefore:

$$\forall g \in \mathcal{G} : b_t(g) = \begin{cases} 1, & \text{if } g \text{ is visited before } t \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

157 $\bullet A = \mathcal{U}_1 \times \dots \times \mathcal{U}_n$ is the set of actions defined as the joint control inputs of n robots $\bullet \mathcal{T} : S \times A \rightarrow S$
 158 is the transition function where at time t for state s_t and action a_t , it is defined as
 159 $\mathcal{T}(s_t, a_t) = (\text{SIM}(s_1, u_1, f_1, dt), \dots, \text{SIM}(s_n, u_n, f_n, dt), b_{t+dt})$ where $s_t = (s_1, \dots, s_n, b_t)$,
 160 $a_t = (u_1, \dots, u_n)$, and $f_1, \dots, f_n$ are the dynamics of the n robots. The b_{t+dt} encodes all the goal
 161 positions reached by the robots at time $t + dt$. $\bullet R : S \times A \rightarrow \mathbb{R}$ is the reward function, which is

the sum of three different functions $R_C, R_G, R_V : S \times A \rightarrow \mathbb{R}$ (designed for the three constraints of equation (6)) that at time t satisfy:

$$R(s_t, a_t) = R_C(s_t, a_t) + R_G(s_t, a_t) + R_V(s_t, a_t) \quad (11)$$

Let $r_s, r_l \in \mathbb{R}^+$ be some arbitrarily fixed small and large rewards where $r_s < r_l$. Let r_M be a very large reward such that $r_M \gg nr_l T_{\max}$ where $T_{\max}$ is the maximum episode length. The first reward function R_C determines the reward for avoiding collisions:

$$R_C(s_t, a_t) = \begin{cases} -r_M, & \text{if } \exists i \in \{1, \dots, n\} : s_i = (q_i, v_i) \wedge \\ & q_i \notin Q_i^{\text{free}}(t) \wedge v_i \in \mathcal{V}_i \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

The second reward function R_G specifies the reward the robots receive for visiting any region of interest (ROI). Let $\mathcal{G}_{t+dt}^{\text{new}}$ be the newly visited ROIs by taking an action a_t from state s_t :

$$\mathcal{G}_{t+dt}^{\text{new}} = \{g \in \mathcal{G} \mid b_t(g) = 0 \wedge b_{t+dt}(g) = 1\} \quad (13)$$

Therefore:

$$R_G(s_t, a_t) = \begin{cases} +r_M, & \text{if } \forall g \in \mathcal{G} : b_{t+dt}(g) = 1 \\ r_l \cdot |\mathcal{G}_{t+dt}^{\text{new}}|, & \text{otherwise} \end{cases} \quad (14)$$

The third reward function R_V determines the reward that the robots receive by checking whether the locations in the state s_t were visited previously in the episode. Let $V : \mathcal{W} \rightarrow \{0, 1\}$ be a shared occupancy map where $V(x) = 1$ if the location $x \in \mathcal{W}$ is visited before t in the current episode and 0 otherwise. We define the number of locations in the workspace that were visited by the n robots before t in the current episode as:

$$n_{v_t} = \sum_{i=1}^n \{i \mid \exists x \in \mathcal{P}_i(s_i) : V(x) = 1\} \quad (15)$$

Therefore:

$$R_V(s_t, a_t) = -r_l n_{v_t} - r_s (n - n_{v_t}) \quad (16)$$

• $\gamma \in [0, 1]$ is the discount factor that is applied to future rewards. For an infinite time horizon ($T = \infty$), the discount factor γ must be less than 1 to converge $\sum_{t \geq 0} \gamma^t r_t$ to a finite value when measuring the Q-value of a state and action. But for a finite time horizon ($T < \infty$), $\gamma = 1$ means that the reward for each timestep doesn't get discounted, and the cumulative reward is still finite and valid. This is a valid assumption since robots have power constraints and can only run for a limited time. Let $T_1, \dots, T_n$ be the maximum runtimes of the n robots. Therefore, we take the maximum length of each finite episode to be $T_{\max} = \min(T_1, \dots, T_n)$. As discussed by Sutton and Barto [10], the return objective takes future rewards into account more strongly as γ approaches 1. Based on this discussion, we define the following theorem:

Theorem 1 For a finite time horizon T , the optimal policy π^* for the MDP M will generate the paths that solve the optimization problem described in equation (6) if $\gamma = 1$.

Proof: The definition of optimal policy π^* and optimal action-value function Q^{π^*} [10] are as follows:

$$\pi^*(s_t) = \arg \max_a Q^{\pi^*}(s_t, a_t) \quad (17)$$

$$\begin{aligned} Q^{\pi^*}(s_t, a_t) &= \max_{\pi} Q^{\pi}(s_t, a_t) \\ &= \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s_t, a_0 = a_t, \pi \right] \\ &= \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R(s_t, a_t) \mid s_0 = s_t, a_0 = a_t, \pi \right] \end{aligned} \quad (18)$$

$$[r_t = R(s_t, a_t)]$$

$$= \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t (R_C(s_t, a_t) + R_G(s_t, a_t) + R_V(s_t, a_t)) \right]$$

$$| s_0 = s_t, a_0 = a_t, \pi] \text{ [From equation (11)]}$$

189 From equation (12), the maximum value of $R_C(s_t, a_t)$ is zero, which by definition means there are
 190 no collisions between any two robots or a robot and an obstacle, satisfying the second constraint of
 191 equation (6). The maximum value of $R_G(s_t, a_t)$ is r_M (since $r_M \gg nr_l T_{max}$), which means all the
 192 regions of interest are visited, satisfying the first constraint of equation (6). From the definition of
 193 the transition function:

$$\begin{aligned} \mathcal{T}(s_t, a_t) &= (\text{SIM}(s_1, u_1, f_1, dt), \dots, \text{SIM}(s_n, u_n, f_n, dt), b_{t+dt}) \\ &= (\text{SIM}(\zeta_1(t), u_1, f_1, dt), \dots, \text{SIM}(\zeta_n(t), u_n, f_n, dt), b_{t+dt}) \\ &= (\zeta_1(t+dt), \dots, \zeta_n(t+dt), b_{t+dt}) \end{aligned} \quad (19)$$

194 This satisfies the third constraint of equation (6). For a finite time horizon T (each episode), we have
 195 the optimal action-value function:

$$Q^{\pi^*}(s_t, a_t) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t (R_C(s_t, a_t) + R_G(s_t, a_t) + R_V(s_t, a_t)) \mid \cdot \right]$$

$$= 0 + r_M + \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t (R_V(s_t, a_t)) \mid \cdot \right]$$

[From the above discussion]

$$= r_M + \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T (-r_l n_{v_t} - r_s (n - n_{v_t})) \mid \cdot \right]$$

[From equation (16) and $\gamma^t = 1$]

$$= r_M + \min_{\pi} \mathbb{E} \left[\sum_{t=0}^T (r_l n_{v_t} + r_s (n - n_{v_t})) \mid \cdot \right] \quad (20)$$

$$[\max_{a \in A} (-f(a)) = \min_{a \in A} (f(a))]$$

196 From equation (6), let $\mathcal{C}_i(\zeta_i(t)) = \mathcal{J}_i(t)$ be the cost for robot i at time t . Since the robots can only
 197 be in one state $s \in \mathcal{S}_i$ at a time t , $\zeta^{-1}(s)$ will return the time that the robot was at that state if it was
 198 visited, and false otherwise. $\mathcal{C}^{-1}(c_0)$ and $\mathcal{C}^{-1}(c_1)$ will return the set of states that has the costs c_0 and
 199 c_1 respectively, and each set is a subset of $\mathcal{S}_i$. Therefore, $\mathcal{J}_i(t)$ or $\mathcal{C}_i(\zeta_i(t))$ is a measurable function.
 200 From the theorems of Fubini and Tonelli [39], we have $\sum_a \int_b f(a, b) db = \int_b \sum_a f(a, b) db$ if the
 201 total sum is finite and $f(a, b)$ is measurable. Therefore from equation (6):

$$\begin{aligned} \min \left(\sum_{i=1}^n \int_{t=0}^T \mathcal{C}_i(\zeta_i(t)) dt \right) &= \min \left(\int_{t=0}^T \sum_{i=1}^n \mathcal{C}_i(\zeta_i(t)) dt \right) \\ &= \min \left(\int_{t=0}^T \left(\sum_{i=1}^{n_{v_t}} \mathcal{C}_i(\zeta_i(t)) + \sum_{i=n_{v_t}+1}^n \mathcal{C}_i(\zeta_i(t)) \right) dt \right) \\ &= \min \left(\sum_{i=1}^{n_{v_t}} \int_{t=0}^T c_1 dt + \sum_{i=n_{v_t}+1}^n \int_{t=0}^T c_0 dt \right) \\ &= \min \left(\sum_{i=1}^{n_{v_t}} c_1 T + \sum_{i=n_{v_t}+1}^n c_0 T \right) \end{aligned}$$

$$= \min(c_1 n_{v_t} T + c_0 (n - n_{v_t}) T) \quad (21)$$

202 Since by definition $c_0 < c_1$ and $r_s < r_l$, let $c_0 = r_s$ and $c_1 = r_l$ without loss of generality.
 203 Therefore:

$$\begin{aligned} & \min(c_1 n_{v_t} T + c_0 (n - n_{v_t}) T) \\ &= \min(r_l n_{v_t} T + r_s (n - n_{v_t}) T) \\ &= \min\left(\sum_{t=1}^T (r_l n_{v_t} + r_s (n - n_{v_t}))\right) \quad \left[\sum_{i=1}^n x = nx\right] \\ &\equiv r_M + \min_{\pi} \mathbb{E}\left[\sum_{t=0}^T (r_l n_{v_t} + r_s (n - n_{v_t})) \mid \cdot\right] \\ &\equiv Q^{\pi^*}(s_t, a_t) \end{aligned} \quad (22)$$

204 Here, the $\equiv$ symbol means that they have a one-to-one mapping or bijection (since r_M is a constant
 205 and the reward distribution is fixed). Hence, the optimal cost of navigation within the time horizon
 206 T is equivalent to the optimal action-value $Q^{\pi^*}(s_t, a_t)$. Therefore, the optimal policy π^* for the
 207 given MDP M will generate the optimal paths with minimum costs, satisfying all three constraints
 208 of equation (6). $\square$

209 **Remark (software versus idealized MDP).** The proof above analyzes the deterministic MDP in
 210 which $\text{SIM}(\cdot)$ follows the nominal dynamics and the agent observes the true state. Our public Gym-
 211 nasium implementation adds (i) stochastic wind draws and action noise inside the transition, (ii)
 212 Gaussian observation noise on the policy inputs, (iii) optional domain randomization of physical
 213 parameters between episodes, and (iv) additional dense reward terms that improve empirical learn-
 214 ability. Consequently, finite-sample RL runs should be interpreted as approximating the idealized
 215 objective rather than satisfying Theorem 1 symbol-for-symbol. Tightening the theory to the de-
 216 ployed reward and stochastic kernel is left for future work; meanwhile, the idealized MDP clarifies
 217 constraint structure and informs reward scaling.

218 6 Experiments

219 6.1 Robot Dynamics

220 Experimental evaluations were conducted for two groups of robots with different dynamics. One
 221 group consists of n wheeled robots operating in an environment with obstacles [?], and the other
 222 group consists of n omnidirectional UAVs operating in a windy environment. The dynamics of the
 223 robots are described as follows:

224 6.1.1 Wheeled robots with obstacles

225 The configuration $(x_i, y_i, \theta_i) \in \mathcal{Q}_i$ for robot i has the position (x_i, y_i) and orientation θ_i . The state
 226 $(x_i, y_i, \theta_i, \psi_i, v_i) \in \mathcal{S}_i$ includes the steering angle ψ_i and velocity v_i . The control $(\alpha, \omega) \in \mathcal{U}_i$
 227 defines the acceleration α and steering rate ω . The motion dynamics are defined as:

$$\dot{s} = f_i(s, u) = (\dot{x}, \dot{y}, \dot{\theta}, \dot{\psi}, \dot{v}), \quad (23)$$

$$\dot{x} = v \cos(\theta) \cos(\psi), \quad \dot{y} = v \sin(\theta) \cos(\psi), \quad (24)$$

$$\dot{\theta} = v \sin(\psi) / L, \quad \dot{v} = \alpha, \quad \dot{\psi} = \omega, \quad (25)$$

228 where L is the distance from the back to the front wheels. In the software, planar wind drift and
 229 actuator noise can be injected so that realized motions deviate from the nominal field shown above.

230 6.1.2 UAVs in windy environment

231 The configuration $(x_i, y_i) \in \mathcal{Q}_i$ denotes the center position of robot i , and state $(x_i, y_i, v_{x_i}, v_{y_i}) \in$
 232 $\mathcal{S}_i$ denotes the position and velocities along the x and y axis respectively. The control $(\mu_{x_i}, \mu_{y_i}) \in$

233 $\mathcal{U}_i$ denotes the forces applied along the x and y axis to change its velocity. For the obstacle-free
 234 aerial layouts used in training, $\cup_j \mathcal{Q}_i \mathcal{O}_j = \emptyset$ and $\mathcal{Q}_i^{\text{free}} = \mathcal{Q}_i$. If the wind is blowing at a nominal
 235 angle β_a and magnitude v_a , the *nominal* continuous-time dynamics are

$$\dot{s} = f_i(s, u) = (\dot{x}, \dot{y}, \dot{v}_x, \dot{v}_y), \quad (26)$$

$$\dot{v}_x = \mu_x/M_i, \quad \dot{v}_y = \mu_y/M_i, \quad (27)$$

$$\dot{x} = v_x + v_a \cos(\beta_a), \quad \dot{y} = v_y + v_a \sin(\beta_a), \quad (28)$$

236 where M_i is the mass of robot i . At each step the deployed simulator may resample wind magnitude
 237 and heading around (v_a, β_a) , add action noise before integration, and clip speeds for numerical
 238 stability.

239 6.2 Methodology

240 All experiments were performed on a GPU server with Intel[®] Xeon[®] Silver 4215 CPU @2.50 GHz
 241 and eight NVIDIA[®] A40 GPUs. We crafted **ten** aerial field configurations and **eight** wheeled-
 242 robot configurations with different field shapes, robot placements, and ROI locations (JSON layouts
 243 for UAVs and INI files for wheeled robots in the accompanying repository). We used the Stable-
 244 Baselines3 and SB3-Contrib [40] Python libraries for measuring the performance of six deep RL
 245 algorithms: Advantage Actor-Critic (A2C) [15], Trust Region Policy Optimization (TRPO) [16],
 246 Proximal Policy Optimization (PPO) [17], Truncated Mixture of Continuous Distributional Quantile
 247 Critics (TQC) [18], CrossQ [19], and Augmented Random Search (ARS) [8]. We chose both on-
 248 policy (A2C, TRPO, PPO, ARS) and off-policy (CrossQ, TQC) RL methods for our experiments.
 249 The objective is to compare the learning efficiency of these RL algorithms for different environ-
 250 mental variations in a variety of training conditions. Their performances are measured based on
 251 cumulative rewards over each episode. Unless otherwise noted, each training job uses on the order
 252 of 2×10^6 environment steps per algorithm–layout–seed configuration in the public scripts; shorter
 253 budgets may be used during hyperparameter search. Henderson *et al.*[21] emphasized evaluating
 254 RL methods on a variety of hyperparameters. Therefore, we performed experiments in two settings:
 255 one using the default hyperparameters given in Stable-Baselines3 and SB3-Contrib, and another us-
 256 ing hyperparameters selected by the Optuna [20] library. Hence, for the two robot groups, we have
 257 four primary experiment sets: UAVs with default and tuned hyperparameters, and wheeled robots
 258 with default and tuned hyperparameters. We selected both common and model-specific hyperpa-
 259 rameters for optimization. For a specific hyperparameter, we used the same range for all algorithms
 260 for consistency. The learning rate $\alpha \in [0.0001 - 0.05]$ is used by all algorithms. The generalized
 261 advantage estimator $\lambda \in [0.90 - 1.0]$ is used for all algorithms other than ARS and TQC. The
 262 value function coefficient $\text{vf_coef} \in [0.2 - 0.7]$, entropy coefficient $\text{ent_coef} \in [0.0 - 0.05]$ and
 263 the maximum gradient clipping $\text{max_grad_norm} \in [0.30 - 0.99]$ are only used in A2C, PPO and
 264 CrossQ. The $\text{buffer_size} \in [1000 - 100000]$ is used on all off-policy algorithms (CrossQ, TQC).
 265 We performed 50 different trials in Optuna [20] for 0.5 million timesteps. **Additional switches**
 266 in the codebase enable reward ablations, observation ablations, uncertainty ablations, and domain
 267 randomization; we summarize the most representative main experiments here and defer exhaustive
 268 sweeps to supplementary material.

269 6.3 Results and Analysis

270 Table 1 shows the results of the four primary experiment sets. The mean and maximum reward for
 271 each experiment are highlighted in bold. The range is the absolute difference between the minimum
 272 and maximum reward over the timesteps. The average reward threshold for all the experiments
 273 for finding the optimal policy is 0.1×10^6 . The threshold is computed as the mean episodic reward
 274 averaged over all the environment variations. For UAVs with default hyperparameters, Fig. ?? shows
 275 the performance of each algorithm, where it is evident that CrossQ outperformed all other algorithms
 276 by reaching the reward threshold within 500k timesteps. TRPO, PPO, and TQC exhibit similar
 277 performances, and ARS appears to be improving gradually. A2C was stuck in a local optimum
 278 lower than the others. The maximum reward for TRPO is slightly higher than that of CrossQ,

which confirms that CrossQ was also stuck on a local optimum very close to the global optimal reward. For UAVs with random hyperparameters, Fig. ?? shows the performance of each algorithm, where TRPO and CrossQ achieved comparable maximum rewards. TRPO, PPO, and A2C were robust against changes in hyperparameters, and ARS performed the worst. For wheeled robots with default hyperparameters, Fig. ?? shows the performance of each algorithm. Here, CrossQ was slightly better than ARS, but ARS was more stable and had less variance. A2C had slightly worse performance and was stuck in a local optimum close to the global optimum. For wheeled robots with random hyperparameters, Fig ?? shows that ARS had the best performance, highest stability, and was robust against changes in hyperparameters. It is well-known that PPO can be strong in highly stochastic benchmarks [17, 21]. Our training runs, however, combine structured dynamics mismatch between platforms with multiplicative wind noise, actuator noise, and (optionally) noisy observations; under this mixture, we still observe competitive TRPO and CrossQ performance in several regimes, suggesting that inductive biases of the algorithms interact nontrivially with the environment’s information structure and reward scaling. We caution that episodic returns aggregate dense shaping terms present in the software, so algorithm rankings should be read as empirical sample-complexity comparisons rather than as certificates of optimality for Theorem 1.

6.4 Simulation

We utilized the Gymnasium [22] Python API with Pygame [41] rendering for *both* robot groups during training and evaluation. Additionally, we used CoppeliaSim [42] (previously known as V-Rep [43]) to visualize a 3-D cornfield-style scene in which aerial robots traverse among visually marked target locations; this visualization parallels agricultural patrol routes but should not be confused with the selective *spraying* POMDP studied in our companion paper. Based on the results, the simulations were performed using the trained models for the environment variations 1 and 3 in the scene. Fig. ?? shows the simulation of environment 1 of both robot groups by running inference on the models trained using the CrossQ [19] algorithm. The starting and final positions of the UAVs and target locations (dark green markers) in the CoppeliaSim scene are shown in Fig. ?? and Fig. ??, respectively, where the red lines are executed trajectories covering all targets. The starting and final positions of the wheeled robots in Pygame are shown in Fig. ?? and Fig. ??, respectively, where the obstacles are shown in gray and goal positions are shown in green.

6.5 Sensitivity Analysis

To measure the robustness of the environment design, we ran inference of the UAVs for environment variation 3, where the model was trained using the CrossQ algorithm with no winds ($v_a = 0, \beta_a = 0^\circ$), and the inference was run with the wind blowing with variable speeds v_a and angle $\beta_a = 30^\circ$. Fig. ?? shows the results where the robots were able to find near-optimal visitation trajectories where $v_a \leq 0.4$. The robustness depends on the maximum and minimum speeds of the UAVs by taking any action. We use $v_{clip} = \pm 5$ units per second for the maximum and minimum speed, which prevents a UAV from gaining unreasonable speed that cannot be visualized in the simulators. Therefore, engineers can use the trained model to generate paths in uncertain and windy environments if $-\frac{v_{clip}}{10} < v_a < \frac{v_{clip}}{10}$ holds under the tested conditions.

6.6 Sim-2-Real Analysis

Although the present study focuses on simulation-based validation, the proposed framework is designed to be extensible to real robotic systems. First, the MDP formulation explicitly incorporates robot dynamics through a generic function f_i , enabling direct replacement with system-identified or hardware-calibrated models. Second, environmental disturbances such as wind are modeled within the transition function and varied during inference, serving as a form of structured domain randomization to assess robustness. Additional sensing noise, actuator delays, and model uncertainties can be incorporated directly into the state-transition dynamics without modifying the high-level optimization viewpoint. Furthermore, actuator saturation and finite episode horizons already reflect realistic physical constraints such as battery limits and velocity bounds. Because the environment

Table 1: Experimental results of each algorithm (aggregate statistics; refresh numbers after retraining with ten UAV and eight wheeled layouts).

Experiment	Param.	A2C $\times 10^6$	PPO $\times 10^6$	TRPO $\times 10^6$	ARS $\times 10^6$	CrossQ $\times 10^6$	TQC $\times 10^6$
UAV Default	Mean	-0.5160	-0.2128	-0.0954	-0.2706	0.1429	-0.1977
	Max	-0.1002	-0.0900	0.1594	-0.0802	0.1593	0.1326
	SD	0.2437	0.1846	0.1157	1.4830	0.0990	0.5731
	Range	6.6097	3.7044	1.6792	29.7291	9.9617	21.1364
UAV Random	Mean	-0.2436	-0.1807	-0.1404	-1.5345	-0.0554	-0.8592
	Max	-0.0820	-0.0767	0.0067	-0.1408	0.1593	0.0142
	SD	0.0990	0.0669	0.1011	4.3781	0.3986	2.7137
	Range	2.3568	0.6550	0.6613	19.7610	11.7615	18.5191
Wheeled Default	Mean	-0.0166	-0.0630	-0.0433	-0.0035	0.0847	-0.0673
	Max	0.0193	0.0189	0.0611	0.0196	0.1399	0.0235
	SD	0.0250	0.0362	0.0294	0.0139	0.0838	0.0235
	Range	0.1202	0.1225	0.1638	0.1147	0.2406	0.1251
Wheeled Random	Mean	-0.0960	-0.0963	-0.0448	-0.0053	-0.0830	-0.0718
	Max	-0.0707	-0.0631	0.0163	0.0190	0.1157	0.0144
	SD	0.0045	0.0044	0.0277	0.0123	0.0194	0.0248
	Range	0.0300	0.0385	0.1182	0.1074	0.2170	0.1176

is formally specified as an MDP, it can also be integrated with reachability analysis, control barrier functions, or temporal-logic-based safety verification prior to deployment. While hardware validation remains future work, these design choices provide a principled pathway for bridging the sim-to-real gap. In software, we additionally provide scripts to quantify performance degradation when injecting observation-level perturbations at deployment time.

7 Limitations

This work is an improvement of a previous study where the state and action spaces were discrete and finite, and the dynamics were limited to a specific group of robots. The maximum path or episode length (correlated to flight time) for any robot is fixed and enough for every environment variation to visit all regions of interest. But if the episode length (or flight time) is not enough to visit all locations, then we may get a sub-optimal path, for which further experiments need to be performed. All robots in a group are identical and follow the same dynamics. The sample complexity of the state space increases exponentially as the number of robots increases. We tested our environment with up to seven robots. Even though the learned model can reach the reward threshold within a reasonable time, the absolute maximum reward cannot be guaranteed. The hyperparameters were selected based on their performance in environment variation 1 using Optuna, and therefore are not entirely optimal. Collision avoidance is guaranteed for the *idealized* optimal policy of Section 5 through the reward design and formal proof. Learned policies under function approximation, stochastic transitions, and noisy observations may violate constraints at run time; additional safety verification (e.g., model checking via temporal logic) remains important. Even though the dynamics that we’ve implemented are relatively simple, it is not a limitation since our approach can be used to implement any complex robot dynamics and variable environments. Despite these limitations, we believe that our work will guide researchers on how to improve the implementation of RL in each development stage, from formally defining the MDP to controlling robots in uncertain and noisy environments.

8 Conclusion

We studied learning-based multi-robot navigation in stochastic and uncertain settings while contrasting aerial point-mass dynamics with wheeled bicycle dynamics. An idealized finite-horizon MDP specification and optimization viewpoint clarify constraint structure, while the released Gymnasium software exposes wind, actuator, and sensing noise together with optional domain randomization

357 and ablations for systematic benchmarking. Standard deep RL algorithms train successfully in this
358 setting, and CoppeliaSim visualizations illustrate qualitatively reasonable trajectories under wind
359 perturbations. Future work includes tightening the formal model to match deployed rewards and ker-
360 nels, hardware validation, and extending the companion spraying formulation to coupled ground-air
361 teams.

References

- [1] Y. F. Chen, M. Liu, M. Everett, and J. P. How. Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 285–292, 2017. doi:10.1109/ICRA.2017.7989037.
- [2] M. Everett, Y. F. Chen, and J. P. How. Motion planning among dynamic, decision-making agents with deep reinforcement learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, page 3052–3059. IEEE Press, 2018. doi:10.1109/IROS.2018.8593871. URL <https://doi.org/10.1109/IROS.2018.8593871>.
- [3] P. Long, W. Liu, and J. Pan. Deep-learned collision avoidance policy for distributed multiagent navigation. *IEEE Robotics and Automation Letters*, 2(2):656–663, 2017. doi:10.1109/LRA.2017.2651371.
- [4] T. Fan, P. Long, W. Liu, and J. Pan. Distributed multi-robot collision avoidance via deep reinforcement learning for navigation in complex scenarios. *The International Journal of Robotics Research*, 39(7):856–892, 2020. doi:10.1177/0278364920916531. URL <https://doi.org/10.1177/0278364920916531>.
- [5] R. Han, S. Chen, S. Wang, Z. Zhang, R. Gao, Q. Hao, and J. Pan. Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards. *IEEE Robotics and Automation Letters*, 7(3):5896–5903, 2022. doi:10.1109/LRA.2022.3161699.
- [6] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym, 2016. URL <https://arxiv.org/abs/1606.01540>.
- [7] R. Islam, P. Henderson, M. Gomrokchi, and D. Precup. Reproducibility of benchmarked deep reinforcement learning tasks for continuous control. *arXiv preprint arXiv:1708.04133*, 2017.
- [8] H. Mania, A. Guy, and B. Recht. Simple random search provides a competitive approach to reinforcement learning. *CoRR*, abs/1803.07055, 2018. URL <http://arxiv.org/abs/1803.07055>.
- [9] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, Apr. 2021. ISSN 1573-0565. doi:10.1007/s10994-021-05961-4. URL <http://dx.doi.org/10.1007/s10994-021-05961-4>.
- [10] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [11] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [12] S. Risi and J. Togelius. Neuroevolution in games: State of the art and open challenges. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(1):25–41, 2015.
- [13] K. Kurach, A. Raichuk, P. Stańczyk, M. Zajac, O. Bachem, L. Espeholt, C. Riquelme, D. Vincent, M. Michalski, O. Bousquet, et al. Google research football: A novel reinforcement learning environment. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 4501–4510, 2020.
- [14] H. Zhang, S. Feng, C. Liu, Y. Ding, Y. Zhu, Z. Zhou, W. Zhang, Y. Yu, H. Jin, and Z. Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference, WWW ’19*, page 3620–3624, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450366748. doi:10.1145/3308558.3314139. URL <https://doi.org/10.1145/3308558.3314139>.

- [15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.
- [16] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pages 1889–1897. PMLR, 2015.
- [17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [18] A. Kuznetsov, P. Shvechikov, A. Grishin, and D. Vetrov. Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. In *International conference on machine learning*, pages 5556–5566. PMLR, 2020.
- [19] A. Bhatt, D. Palenicek, B. Belousov, M. Argus, A. Amiranashvili, T. Brox, and J. Peters. Crossq: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International Conference on Learning Representations (ICLR)*, 2024.
- [20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [21] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [22] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [23] N. Ahmed, C. J. Pawase, and K. Chang. Distributed 3-d path planning for multi-uavs with full area surveillance based on particle swarm optimization. *Applied Sciences*, 11(8), 2021. ISSN 2076-3417. doi:10.3390/app11083417. URL <https://www.mdpi.com/2076-3417/11/8/3417>.
- [24] S. S. Ge and C. Fua. Complete multi-robot coverage of unknown environments with minimum repeated coverage. In *Proceedings of the 2005 IEEE International Conference on Robotics and Automation*, pages 715–720, 2005. doi:10.1109/ROBOT.2005.1570202.
- [25] M. Li, A. Richards, and M. Sooriyabandara. Reliability-aware multi-uav coverage path planning using a genetic algorithm. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '21, page 1584–1586, Richland, SC, 2021. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450383073.
- [26] T. Goto, T. Kosaka, and H. Noborio. On the heuristics of a* or a algorithm in its and robot path-planning. In *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, volume 2, pages 1159–1166 vol.2, 2003. doi:10.1109/IROS.2003.1248802.
- [27] Rodriguez, X. Tang, J.-M. Lien, and N. Amato. An obstacle-based rapidly-exploring random tree. In *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, pages 895–900, 2006. doi:10.1109/ROBOT.2006.1641823.
- [28] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller. Anytime motion planning using the rrt*. In *2011 IEEE International Conference on Robotics and Automation*, pages 1478–1483, 2011. doi:10.1109/ICRA.2011.5980479.

- [29] N. Stenderhauf, O. Brock, W. Scheirer, R. Hadsell, D. Fox, J. Leitner, B. Upcroft, P. Abbeel, W. Burgard, M. Milford, and P. Corke. The limits and potentials of deep learning for robotics. *The International Journal of Robotics Research*, 37(4-5):405–420, 2018. doi:10.1177/0278364918770733. URL <https://doi.org/10.1177/0278364918770733>.
- [30] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine. How to train your robot with deep reinforcement learning: lessons we have learned. *The International Journal of Robotics Research*, 40(4-5):698–721, 2021. doi:10.1177/0278364920987859. URL <https://doi.org/10.1177/0278364920987859>.
- [31] Y. Zhang, M. C. Fontaine, V. Bhatt, S. Nikolaidis, and J. Li. Multi-robot coordination and layout design for automated warehousing. *arXiv preprint arXiv:2305.06436*, 2023.
- [32] C. Cai, J. Chen, Q. Yan, and F. Liu. A multi-robot coverage path planning method for maritime search and rescue using multiple auvs. *Remote Sensing*, 15(1):93, 2022.
- [33] H. Osooli. A multi-robot task assignment framework for search and rescue with heterogeneous teams. Master’s thesis, University of Massachusetts Lowell, 2024.
- [34] J. Kober and J. Peters. *Reinforcement Learning in Robotics: A Survey*, pages 9–67. Springer International Publishing, Cham, 2014. ISBN 978-3-319-03194-1. doi:10.1007/978-3-319-03194-1_2. URL https://doi.org/10.1007/978-3-319-03194-1_2.
- [35] H. Bae, G. Kim, J. Kim, D. Qian, and S. Lee. Multi-robot path planning method using reinforcement learning. *Applied Sciences*, 9(15), 2019. ISSN 2076-3417. doi:10.3390/app9153057. URL <https://www.mdpi.com/2076-3417/9/15/3057>.
- [36] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone. Deep reinforcement learning for robotics: a survey of real-world successes. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’25/IAAI’25/EAAI’25. AAAI Press, 2025. ISBN 978-1-57735-897-8. doi:10.1609/aaai.v39i27.35095. URL <https://doi.org/10.1609/aaai.v39i27.35095>.
- [37] T. Tao. *An Introduction to Measure Theory*. American Mathematical Society, Providence, RI, 2011.
- [38] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. The MIT Press, 2005.
- [39] H. L. Royden and P. M. Fitzpatrick. *Real Analysis*. Prentice Hall, Upper Saddle River, NJ, 4th edition, 2010. ISBN 9780131437470. Fourth edition, revised reprint.
- [40] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- [41] Pygame community. Pygame. <https://www.pygame.org/>, 2000-2024. Version 2.1.0.
- [42] E. Rohmer, S. P. N. Singh, and M. Freese. Coppeliasim (formerly v-rep): a versatile and scalable robot simulation framework. In *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013. www.coppeliarobotics.com.
- [43] E. Rohmer, S. P. N. Singh, and M. Freese. V-rep: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326, 2013. doi:10.1109/IROS.2013.6696520.